Parallel Order Book Matching Engine

https://azdeenzack.github.io/parallel-order-book/

Summary

We plan to build a limit order matching engine in C++ and study whether multiple CPU cores can speed up matching within one busy order book. We will compare a sequential engine with fine grained locking, groups of orders that can safely run together, and parallel work within an order that fills many waiting orders. We will run these versions on multicore CPU machines and measure correctness, throughput, latency, and how performance changes with the workload.

Background

What is an Order Book?

A limit order book is a data structure used by financial exchanges to maintain outstanding buy and sell orders for securities. Each order specifies a side (buy or sell), quantity, price, and arrival time and then incoming orders are matched against existing orders according to price time priority, meaning orders offering the best price are executed first, with earlier orders receiving priority when multiple orders share the same price. Depending on available resting orders, an incoming order may be fully executed, partially executed, or added to the book if it cannot be matched at that time.

What are we implementing?

Our project aims to implement an engine for a single security that processes a sequence of limit orders. Then, if we have extra time, we may add market orders and cancellations. We plan to have the engine maintain the order book by tracking the price levels and resting orders, updating the book as new orders arrive, and generating trade executions when compatible orders end up getting matches. We'll first develop a sequential implementation to establish a baseline for correctness and performance before starting to explore parallel implementations. While separate securities can be processed independently, were focusing on identifying opportunities for parallelism within a single shared order book.

Why might parallelism help?

Parallelism within a single order book is possible because some operations may access independent orders or price levels and therefore execute concurrently. Additionally, large incoming orders may interact with multiple resting orders which creates opportunities to parallelize portions of the matching process. We plan to compare fine grained locking, dependency aware batching, and dividing the fills of large incoming orders among workers. We may also explore speculative execution (similar to speculative branching in comparch) if time

permits. However, because incoming orders may interact with shared price levels and affect subsequent executions, their processing is not inherently independent. As a result, parallelizing a single order book requires careful coordination to preserve correct matching behavior rather than simply distributing incoming orders across multiple threads.

The Challenge

Orders for one security share the same book, so they are not automatically independent. Two incoming orders might compete for the same waiting order. One order might use up the best price level, which changes what the next order should match. Orders added at the same price also need to keep their arrival order. These dependencies make it difficult to let several threads change the book at once.

We want to explore three possible sources of parallel work. First, orders that touch separate price levels may be able to proceed at the same time if we coordinate access to the best prices. Second, we may be able to examine a small group of incoming orders and identify which ones are independent before giving them to different workers. Third, one large order that fills many waiting orders may contain enough work to divide among workers. We dont know which approach will work best, or how much of its benefit will be lost to locks, checking dependencies, and combining results, but our project aims to answer this.

Resources

We plan to write the project code in C++. Published work on concurrent order books and parallel prefix sums will help us understand possible methods. We do not need a live market feed or a starter code base for the main experiments, as we will use simulated market feed.

Goals and Deliverables

What we plan to achieve

We will first write a sequential engine with a clear matching rule. It will keep buy and sell orders in price order and keep orders at the same price in arrival order. We will also make a repeatable order generator and a checker that compares the full trade record and final book state. This gives us a way to test whether a parallel version still produces the right answer.

For the first parallel approach, we will try fine grained locking around parts of the book. We will start with orders that do not immediately trade and can update separate prices. Orders that cross the book need more care because the best available price can change. We will compare this method with a simple version that protects the whole book with one lock and measure when the smaller locks help or hurt.

For the second approach, we will examine groups of incoming orders before processing them. A first version can group orders that clearly do not interfere with one another. We will then investigate whether a more careful dependency check lets us include additional orders. We will vary the group size and compare the time saved by running work together with the time spent finding safe groups and waiting for workers.

For the third approach, we will look at an incoming order that fills many waiting orders. We will arrange the eligible waiting orders in the order they should be matched and use a parallel prefix calculation to figure out how much each one fills. We will then update the book and record the trades in the correct order. We plan to include the time needed to prepare that ordered view and apply the results when comparing this approach with the sequential engine.

We will compare all three approaches on the same workloads while varying the number of threads, how often orders cross, how concentrated orders are at a few prices, how many waiting orders get filled, and the size of each group. We will measure total orders processed per second, order latency, and the time spent coordinating workers.

If we have extra time

If we have more time, we could compare fixed and dynamic scheduling when several symbols have very different levels of activity. If the exact matching rules leave little useful work within one book, another option is to define a small, explicit change to the matching rule and measure how much extra parallelism it allows and how the resulting trades differ. We would choose one of these directions based on what we learn from the main experiments.

If work takes longer

If the implementation takes longer than expected, our minimum goal is the sequential engine and checker, two distinct parallel approaches with exact correctness checks, and a detailed analysis of where their time goes. We would use the milestone to update the remaining schedule and focus on the most useful comparison.

Platform Choice

We plan to run on the GHC lab machines. CPU threads are a good fit for a book that changes frequently and has shared state. They also let us compare locking, scheduling, and memory costs directly. We do not need a GPU or multiple machines for the main experiments.

Schedule

Week	What we hope to have done by the end of the week
Week 1 (week of November 9)	- Define the matching rules were going to want - Build the sequential engine and whatever workloads - Check trade records and states of the book
Week 2 (week of November 16)	- Build the fine grain locking and whole book and compare both methods, checking they also both make correct trades - Collect first measurements and prepare the milestone report due November 23
Week 3 (week of November 23)	- Submit the milestone report and update our plan based on the first results including any pivoting needed - Build a grouped order approach that checks which orders can run together - Integrate the parallel large fill approach
Week 4 (week of November 30)	- Compare the three approaches across different workloads and thread counts - Measure time spent on locks, grouping, fill calculations, and ordered trade output - Improve the parts that the measurements identify as bottlenecks or expand
Week 5 (week of December 7)	- Repeat key experiments and prepare figures and conclusions - Finish the final report by December 10 - Prepare and present the poster on December 11